

# Planetaire Mono

(x, a, o) ratio, r, y(ill)ic  
-i-f- (o) f-l-sili-t  
-c-a-i-a-m-l-f-m-a  
rs,000+ i-f-m

2. Amplification of the target DNA sequence  
 using primers and thermostable DNA polymerase  
 (Taq polymerase)

1. la-ai r- f- m- f- B-ys, a- y- r- e- f- i- y- x- a- i- o- i- y-  
 f- i- f- C- C- f- i- f- a- y- e- f- i- C- m- l- a- f- i- C-  
 i- f- a- f- C- r- f- C- t- a- i- y- m- l- a- r- s, o- o- t- a- v- l- f- i- C- f- .

1. The first part of the document is a letter from the author to the editor of the journal. The letter is dated 1968 and is addressed to the editor of the journal. The author is a student of the University of California, Berkeley. The letter is a request for a review of the author's thesis. The author is a student of the University of California, Berkeley. The letter is a request for a review of the author's thesis.

## NOTES

05-1a1 (200) 05-1a2 (200) Medium (200) Medium Italic (200)  
05-1a3 (700) 05-1a4 (700) Extra Bold (800) Extra Bold Italic (800)

**የኢትዮጵያ ፌዴራላዊ ዲሞክራሲያዊ ሪፐብሊክ**

placental transfer variability, but which bears significant correlation with fetal variability at small size and in late gestation (Fig. 1).

1. 7-10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100. 101. 102. 103. 104. 105. 106. 107. 108. 109. 110. 111. 112. 113. 114. 115. 116. 117. 118. 119. 120. 121. 122. 123. 124. 125. 126. 127. 128. 129. 130. 131. 132. 133. 134. 135. 136. 137. 138. 139. 140. 141. 142. 143. 144. 145. 146. 147. 148. 149. 150. 151. 152. 153. 154. 155. 156. 157. 158. 159. 160. 161. 162. 163. 164. 165. 166. 167. 168. 169. 170. 171. 172. 173. 174. 175. 176. 177. 178. 179. 180. 181. 182. 183. 184. 185. 186. 187. 188. 189. 190. 191. 192. 193. 194. 195. 196. 197. 198. 199. 200. 201. 202. 203. 204. 205. 206. 207. 208. 209. 210. 211. 212. 213. 214. 215. 216. 217. 218. 219. 220. 221. 222. 223. 224. 225. 226. 227. 228. 229. 230. 231. 232. 233. 234. 235. 236. 237. 238. 239. 240. 241. 242. 243. 244. 245. 246. 247. 248. 249. 250. 251. 252. 253. 254. 255. 256. 257. 258. 259. 260. 261. 262. 263. 264. 265. 266. 267. 268. 269. 270. 271. 272. 273. 274. 275. 276. 277. 278. 279. 280. 281. 282. 283. 284. 285. 286. 287. 288. 289. 290. 291. 292. 293. 294. 295. 296. 297. 298. 299. 300. 301. 302. 303. 304. 305. 306. 307. 308. 309. 310. 311. 312. 313. 314. 315. 316. 317. 318. 319. 320. 321. 322. 323. 324. 325. 326. 327. 328. 329. 330. 331. 332. 333. 334. 335. 336. 337. 338. 339. 340. 341. 342. 343. 344. 345. 346. 347. 348. 349. 350. 351. 352. 353. 354. 355. 356. 357. 358. 359. 360. 361. 362. 363. 364. 365. 366. 367. 368. 369. 370. 371. 372. 373. 374. 375. 376. 377. 378. 379. 380. 381. 382. 383. 384. 385. 386. 387. 388. 389. 390. 391. 392. 393. 394. 395. 396. 397. 398. 399. 400. 401. 402. 403. 404. 405. 406. 407. 408. 409. 410. 411. 412. 413. 414. 415. 416. 417. 418. 419. 420. 421. 422. 423. 424. 425. 426. 427. 428. 429. 430. 431. 432. 433. 434. 435. 436. 437. 438. 439. 440. 441. 442. 443. 444. 445. 446. 447. 448. 449. 450. 451. 452. 453. 454. 455. 456. 457. 458. 459. 460. 461. 462. 463. 464. 465. 466. 467. 468. 469. 470. 471. 472. 473. 474. 475. 476. 477. 478. 479. 480. 481. 482. 483. 484. 485. 486. 487. 488. 489. 490. 491. 492. 493. 494. 495. 496. 497. 498. 499. 500. 501. 502. 503. 504. 505. 506. 507. 508. 509. 510. 511. 512. 513. 514. 515. 516. 517. 518. 519. 520. 521. 522. 523. 524. 525. 526. 527. 528. 529. 530. 531. 532. 533. 534. 535. 536. 537. 538. 539. 540. 541. 542. 543. 544. 545. 546. 547. 548. 549. 550. 551. 552. 553. 554. 555. 556. 557. 558. 559. 560. 561. 562. 563. 564. 565. 566. 567. 568. 569. 570. 571. 572. 573. 574. 575. 576. 577. 578. 579. 580. 581. 582. 583. 584. 585. 586. 587. 588. 589. 590. 591. 592. 593. 594. 595. 596. 597. 598. 599. 600. 601. 602. 603. 604. 605. 606. 607. 608. 609. 610. 611. 612. 613. 614. 615. 616. 617. 618. 619. 620. 621. 622. 623. 624. 625. 626. 627. 628. 629. 630. 631. 632. 633. 634. 635. 636. 637. 638. 639. 640. 641. 642. 643. 644. 645. 646. 647. 648. 649. 650. 651. 652. 653. 654. 655. 656. 657. 658. 659. 660. 661. 662. 663. 664. 665. 666. 667. 668. 669. 670. 671. 672. 673. 674. 675. 676. 677. 678. 679. 680. 681. 682. 683. 684. 685. 686. 687. 688. 689. 690. 691. 692. 693. 694. 695. 696. 697. 698. 699. 700. 701. 702. 703. 704. 705. 706. 707. 708. 709. 710. 711. 712. 713. 714. 715. 716. 717. 718. 719. 720. 721. 722. 723. 724. 725. 726. 727. 728. 729. 730. 731. 732. 733. 734. 735. 736. 737. 738. 739. 740. 741. 742. 743. 744. 745. 746. 747. 748. 749. 750. 751. 752. 753. 754. 755. 756. 757. 758. 759. 760. 761. 762. 763. 764. 765. 766. 767. 768. 769. 770. 771. 772. 773. 774. 775. 776. 777. 778. 779. 780. 781. 782. 783. 784. 785. 786. 787. 788. 789. 790. 791. 792. 793. 794. 795. 796. 797. 798. 799. 800. 801. 802. 803. 804. 805. 806. 807. 808. 809. 810. 811. 812. 813. 814. 815. 816. 817. 818. 819. 820. 821. 822. 823. 824. 825. 826. 827. 828. 829. 830. 831. 832. 833. 834. 835. 836. 837. 838. 839. 840. 841. 842. 843. 844. 8

[illegible]
$$E \vdash E \vdash C \vdash \cdot C \vdash E \vdash K \vdash I \vdash \cdot Z \vdash E \vdash I \vdash Z \vdash \cdot I \cup K \vdash I \vdash Z \vdash$$

r-ai-f di-tal a-i-f r-flam a-er-yr-yo-. eal  
n-l-x-l-m-i-gäl t-a-dl=er-worl. El -el-  
w-cilla-ti-r-mia feli-a-ill-y bi i. bil amalı ta-ta  
yağı-ş-f-re ça-ta at d-e-n-i.

CREEK

Ξεσπάζω τὴν ψυχ-φθόρα βδελυγμία.

CAIRGIC

Съешь же ещё этих мягких французских булок, да выпей чаю. Широкая электрификация южных губерний даст мощный толчок подъёму сельского хозяйства.

$$p_{\text{la-0+ai}}^{\text{ro}} \text{ H}^{\text{---}} \cdot \text{D}_{\text{it+ub}} \cdot \text{C}_{\text{-m/ll}}^{\text{ro}} / p_{\text{la-0+ai}}^{\text{ro}} \cdot \text{oe r-v.v}$$

$$p_{\text{la-0+ai}}^{\text{R0}} \text{ W---} . \text{ di+tu+} . \text{ C-m/ll0y/rla-0+ai}^{\text{R0}} . \text{ oEr-y.y}$$

# PyTorch vs GPT: The Complete Algorithm

The most atomic way to train and run inference for a GPT in pure, dependency-free Python.  
This file is the complete algorithm.  
Everything else is just efficiency.

```
"""
The most atomic way to train and run inference for a GPT in pure, dependency-free Python.
This file is the complete algorithm.
Everything else is just efficiency.

@karpathy
"""

import os      # os.path.exists
import math    # math.log, math.exp
import random  # random.seed, random.choices, random.gauss, random.shuffle
random.seed(42) # Let there be order among chaos

# Let there be a Dataset `docs`: List[str] of documents (e.g. a list of names)
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")

# Let there be a Tokenizer to translate strings to sequences of integers ("tokens") and back
uchars = sorted(set(''.join(docs))) # unique characters in the dataset become token ids 0..n-1
BOS = len(uchars) # token id for a special Beginning of Sequence (BOS) token
vocab_size = len(uchars) + 1 # total number of unique tokens, +1 is for BOS
print(f"vocab size: {vocab_size}")

# Let there be Autograd to recursively apply the chain rule through a computation graph
class Value:
    __slots__ = ('data', 'grad', '_children', '_local_grads') # Python optimization for memory usage

    def __init__(self, data, children=(), local_grads=()):
        self.data = data # scalar value of this node calculated during forward pass
        self.grad = 0 # derivative of the loss w.r.t. this node, calculated in backward pass
        self._children = children # children of this node in the computation graph
        self._local_grads = local_grads # local derivative of this node w.r.t. its children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def __pow__(self, other): return Value(self.data**other, (self,), (other * self.data**(other-1),))
    def log(self): return Value(math.log(self.data), (self,), (1/self.data,))
    def exp(self): return Value(math.exp(self.data), (self,), (math.exp(self.data),))
    def relu(self): return Value(max(0, self.data), (self,), (float(self.data > 0),))
    def __neg__(self): return self * -1
    def __radd__(self, other): return self + other
    def __sub__(self, other): return self + (-other)
    def __rsub__(self, other): return other + (-self)
    def __rmul__(self, other): return self * other
    def __truediv__(self, other): return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
            topo.append(v)
        build_topo(self)
        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._local_grads):
                child.grad += local_grad * v.grad
```

```

# Initialize the parameters, to store the knowledge of the model
n_layer = 1      # depth of the transformer neural network (number of layers)
n_embd = 16      # width of the network (embedding dimension)
block_size = 16  # maximum context length of the attention window (note: the longest name is 15 characters)
n_head = 4       # number of attention heads
head_dim = n_embd // n_head # derived dimension of each head
matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0, std)) for _ in range(nin)] for _ in range(nout)]
state_dict = {'wte': matrix(vocab_size, n_embd), 'wpe': matrix(block_size, n_embd), 'lm_head': matrix(vocab_size,
n_embd)}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_embd)
params = [p for mat in state_dict.values() for row in mat for p in row] # flatten params into a single list[Value]
print(f"num params: {len(params)}")

# Define the model architecture: a function mapping tokens and parameters to logits over what comes next
# Follow GPT-2, blessed among the GPTs, with minor differences: layernorm -> rmsnorm, no biases, GeLU -> ReLU
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(w, x)) for wo in w]

def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]

def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]

def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id] # token embedding
    pos_emb = state_dict['wpe'][pos_id] # position embedding
    x = [t + p for t, p in zip(tok_emb, pos_emb)] # joint token and position embedding
    x = rmsnorm(x) # note: not redundant due to backward pass via the residual connection

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head_dim)) / head_dim**0.5 for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t in range(len(v_h))) for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits

# Let there be Adam, the blessed optimizer and its buffers
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params) # first moment buffer
v = [0.0] * len(params) # second moment buffer

# Repeat in sequence
num_steps = 1000 # number of training steps
for step in range(num_steps):

    # Take single document, tokenize it, surround it with BOS special token on both sides

```

```

doc = docs[step % len(docs)]
tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
n = min(block_size, len(tokens) - 1)

# Forward the token sequence through the model, building up the computation graph all the way to the loss
keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
losses = []
for pos_id in range(n):
    token_id, target_id = tokens[pos_id], tokens[pos_id + 1]
    logits = gpt(token_id, pos_id, keys, values)
    probs = softmax(logits)
    loss_t = -probs[target_id].log()
    losses.append(loss_t)
loss = (1 / n) * sum(losses) # final average loss over the document sequence. May yours be low.

# Backward the loss, calculating the gradients with respect to all model parameters
loss.backward()

# Adam optimizer update: update the model parameters based on the corresponding gradients
lr_t = learning_rate * (1 - step / num_steps) # linear learning rate decay
for i, p in enumerate(params):
    m[i] = beta1 * m[i] + (1 - beta1) * p.grad
    v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
    m_hat = m[i] / (1 - beta1 ** (step + 1))
    v_hat = v[i] / (1 - beta2 ** (step + 1))
    p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
    p.grad = 0

print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.data:.4f}", end='\r')

# Inference: may the model babble back to us
temperature = 0.5 # in (0, 1], control the "creativity" of generated text, low to high
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=[p.data for p in probs])[0]
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {' '.join(sample)}")

```



የኢትዮጵያ ፌዴራላዊ ዲሞክራሲያዊ ሪፐብሊክ

Basic rule on behavior:  $r_m$  Boys

В С D E E C I J K Г О Ё О Ё Z I O / N X A T

BASIC RULES FOR EBC:  $\tau$ -m Bets

a - C - D - E - F - G - H - I - J - K - L - M - N - O - P - Q - R - S - T - U - V - W - X - Y -

DECLASS: 0-0 (R-m Berys (-0r- m-difiid With Co-or dat R o/o di'amli)-a-ti-n)

$$0 \leq \alpha \leq 1 \Rightarrow \alpha \in [0, 1]$$

Богородица:  $\int_{-m}^m a(x)$

! " # \$ % & ' ( ) \* + , - . / : ; < = > ? @ [ \ ] ^ \_  
` { | } ~

EXTENDED TITLE: Form Bays

◆ ◆ ◆ ◆ ◆ Å Æ Ç È É Ê Ë Ì Í Î Ï Ð Ñ Ò Ó Ô Õ Ö Ø Ù Ú Û Ü Ý Þ ß à á â ã ä å æ ç ƒ ǁ ǂ ǃ Ǆ ı ı̇ ı̈ ı̉ ı̊ ı̋ ı̌ ı̍ ı̎ ı̏ ı̐ ı̑ ı̒ ı̓ ı̔ ı̕ ı̖ ı̗ ı̘ ı̙ ı̚ ı̛ ı̜ ı̝ ı̞ ı̟ ı̠ ı̡ ı̢ ı̣ ı̤ ı̥ ı̦ ı̧ ı̨ ı̩ ı̪ ı̫ ı̬ ı̭ ı̮ ı̯ ı̰ ı̱ ı̲ ı̳ ı̴ ı̵ ı̶ ı̷ ı̸ ı̹ ı̺ ı̻ ı̼ ı̽ ı̾ ı̿ ı̺̃ ı̻̃ ı̼̃ ı̽̃ ı̾̃ ı̿̃ ı̺̄ ı̻̄ ı̼̄ ı̽̄ ı̾̄ ı̿̄ ı̺̅ ı̻̅ ı̼̅ ı̽̅ ı̾̅ ı̿̅ ı̺̆ ı̻̆ ı̼̆ ı̽̆ ı̾̆ ı̿̆ ı̺̇ ı̻̇ ı̼̇ ı̽̇ ı̾̇ ı̿̇ ı̺̈ ı̻̈ ı̼̈ ı̽̈ ı̾̈ ı̿̈ ı̺̉ ı̻̉ ı̼̉ ı̽̉ ı̾̉ ı̿̉ ı̺̊ ı̻̊ ı̼̊ ı̽̊ ı̾̊ ı̿̊ ı̺̋ ı̻̋ ı̼̋ ı̽̋ ı̾̋ ı̿̋ ı̺̌ ı̻̌ ı̼̌ ı̽̌ ı̾̌ ı̿̌ ı̺̍ ı̻̍ ı̼̍ ı̽̍ ı̾̍ ı̿̍ ı̺̎ ı̻̎ ı̼̎ ı̽̎ ı̾̎ ı̿̎ ı̺̏ ı̻̏ ı̼̏ ı̽̏ ı̾̏ ı̿̏ ı̺̐ ı̻̐ ı̼̐ ı̽̐ ı̾̐ ı̿̐ ı̺̑ ı̻̑ ı̼̑ ı̽̑ ı̾̑ ı̿̑ ı̺̒ ı̻̒ ı̼̒ ı̽̒ ı̾̒ ı̿̒ ı̺̓ ı̻̓ ı̼̓ ı̽̓ ı̾̓ ı̿̓ ı̺̔ ı̻̔ ı̼̔ ı̽̔ ı̾̔ ı̿̔ ı̺̕ ı̻̕ ı̼̕ ı̽̕ ı̾̕ ı̿̕ ı̺̖ ı̻̖ ı̼̖ ı̖̽ ı̖̾ ı̖̿ ı̺̗ ı̻̗ ı̼̗ ı̗̽ ı̗̾ ı̗̿ ı̺̘ ı̻̘ ı̼̘ ı̘̽ ı̘̾ ı̘̿ ı̺̙ ı̻̙ ı̼̙ ı̙̽ ı̙̾ ı̙̿ ı̺̚ ı̻̚ ı̼̚ ı̽̚ ı̾̚ ı̿̚ ı̛̺ ı̛̻ ı̛̼ ı̛̽ ı̛̾ ı̛̿ ı̺̜ ı̻̜ ı̼̜ ı̜̽ ı̜̾ ı̜̿ ı̺̝ ı̻̝ ı̼̝ ı̝̽ ı̝̾ ı̝̿ ı̺̞ ı̻̞ ı̼̞ ı̞̽ ı̞̾ ı̞̿ ı̺̟ ı̻̟ ı̼̟ ı̟̽ ı̟̾ ı̟̿ ı̺̠ ı̻̠ ı̼̠ ı̠̽ ı̠̾ ı̠̿ ı̡̺ ı̡̻ ı̡̼ ı̡̽ ı̡̾ ı̡̿ ı̢̺ ı̢̻ ı̢̼ ı̢̽ ı̢̾ ı̢̿ ı̺̣ ı̻̣ ı̼̣ ı̣̽ ı̣̾ ı̣̿ ı̺̤ ı̻̤ ı̼̤ ı̤̽ ı̤̾ ı̤̿ ı̺̥ ı̻̥ ı̼̥ ı̥̽ ı̥̾ ı̥̿ ı̺̦ ı̻̦ ı̼̦ ı̦̽ ı̦̾ ı̦̿ ı̧̺ ı̧̻ ı̧̼ ı̧̽ ı̧̾ ı̧̿ ı̨̺ ı̨̻ ı̨̼ ı̨̽ ı̨̾ ı̨̿ ı̺̩ ı̻̩ ı̼̩ ı̩̽ ı̩̾ ı̩̿ ı̺̪ ı̻̪ ı̼̪ ı̪̽ ı̪̾ ı̪̿ ı̺̫ ı̻̫ ı̼̫ ı̫̽ ı̫̾ ı̫̿ ı̺̬ ı̻̬ ı̼̬ ı̬̽ ı̬̾ ı̬̿ ı̺̭ ı̻̭ ı̼̭ ı̭̽ ı̭̾ ı̭̿ ı̺̮ ı̻̮ ı̼̮ ı̮̽ ı̮̾ ı̮̿ ı̺̯ ı̻̯ ı̼̯ ı̯̽ ı̯̾ ı̯̿ ı̺̰ ı̻̰ ı̼̰ ı̰̽ ı̰̾ ı̰̿ ı̺̱ ı̻̱ ı̼̱ ı̱̽ ı̱̾ ı̱̿ ı̺̲ ı̻̲ ı̼̲ ı̲̽ ı̲̾ ı̲̿ ı̺̳ ı̻̳ ı̼̳ ı̳̽ ı̳̾ ı̳̿ ı̴̺ ı̴̻ ı̴̼ ı̴̽ ı̴̾ ı̴̿ ı̵̺ ı̵̻ ı̵̼ ı̵̽ ı̵̾ ı̵̿ ı̶̺ ı̶̻ ı̶̼ ı̶̽ ı̶̾ ı̶̿ ı̷̺ ı̷̻ ı̷̼ ı̷̽ ı̷̾ ı̷̿ ı̸̺ ı̸̻ ı̸̼ ı̸̽ ı̸̾ ı̸̿ ı̺̹ ı̻̹ ı̼̹ ı̹̽ ı̹̾ ı̹̿ ı̺̺ ı̻̺ ı̼̺ ı̺̽ ı̺̾ ı̺̿ ı̺̻ ı̻̻ ı̼̻ ı̻̽ ı̻̾ ı̻̿ ı̺̼ ı̻̼ ı̼̼ ı̼̽ ı̼̾ ı̼̿ ı̺̽ ı̻̽ ı̼̽ ı̽̽ ı̾̽ ı̿̽ ı̺̾ ı̻̾ ı̼̾ ı̽̾ ı̾̾ ı̿̾ ı̺̿ ı̻̿ ı̼̿ ı̽̿ ı̾̿ ı̿̿ ı̺̺̃ ı̻̺̃ ı̼̺̃ ı̺̽̃ ı̺̾̃ ı̺̿̃ ı̺̺̄ ı̻̺̄ ı̼̺̄ ı̺̽̄ ı̺̾̄ ı̺̿̄ ı̺̺̅ ı̻̺̅ ı̼̺̅ ı̺̽̅ ı̺̾̅ ı̺̿̅ ı̺̺̆ ı̻̺̆ ı̼̺̆ ı̺̽̆ ı̺̾̆ ı̺̿̆ ı̺̺̇ ı̻̺̇ ı̼̺̇ ı̺̽̇ ı̺̾̇ ı̺̿̇ ı̺̺̈ ı̻̺̈ ı̼̺̈ ı̺̽̈ ı̺̾̈ ı̺̿̈ ı̺̺̉ ı̻̺̉ ı̼̺̉ ı̺̽̉ ı̺̾̉ ı̺̿̉ ı̺̺̊ ı̻̺̊ ı̼̺̊ ı̺̽̊ ı̺̾̊ ı̺̿̊ ı̺̺̋ ı̻̺̋ ı̼̺̋ ı̺̽̋ ı̺̾̋ ı̺̿̋ ı̺̺̌ ı̻̺̌ ı̼̺̌ ı̺̽̌ ı̺̾̌ ı̺̿̌ ı̺̺̍ ı̻̺̍ ı̼̺̍ ı̺̽̍ ı̺̾̍ ı̺̿̍ ı̺̺̎ ı̻̺̎ ı̼̺̎ ı̺̽̎ ı̺̾̎ ı̺̿̎ ı̺̺̏ ı̻̺̏ ı̼̺̏ ı̺̽̏ ı̺̾̏ ı̺̿̏ ı̺̺̐ ı̻̺̐ ı̼̺̐ ı̺̽̐ ı̺̾̐ ı̺̿̐ ı̺̺̑ ı̻̺̑ ı̼̺̑ ı̺̽̑ ı̺̾̑ ı̺̿̑ ı̺̺̒ ı̻̺̒ ı̼̺̒ ı̺̽̒ ı̺̾̒ ı̺̿̒ ı̺̺̓ ı̻̺̓ ı̼̺̓ ı̺̽̓ ı̺̾̓ ı̺̿̓ ı̺̺̔ ı̻̺̔ ı̼̺̔ ı̺̽̔ ı̺̾̔ ı̺̿̔ ı̺̺̕ ı̻̺̕ ı̼̺̕ ı̺̽̕ ı̺̾̕ ı̺̿̕ ı̺̺̖ ı̻̺̖ ı̼̺̖ ı̺̖̽ ı̺̖̾ ı̺̖̿ ı̺̺̗ ı̻̺̗ ı̼̺̗ ı̺̗̽ ı̺̗̾ ı̺̗̿ ı̺̺̘ ı̻̺̘ ı̼̺̘ ı̺̘̽ ı̺̘̾ ı̺̘̿ ı̺̺̙ ı̻̺̙ ı̼̺̙ ı̺̙̽ ı̺̙̾ ı̺̙̿ ı̺̺̚ ı̻̺̚ ı̼̺̚ ı̺̽̚ ı̺̾̚ ı̺̿̚ ı̛̺̺ ı̛̻̺ ı̛̼̺ ı̛̺̽ ı̛̺̾ ı̛̺̿ ı̺̺̜ ı̻̺̜ ı̼̺̜ ı̺̜̽ ı̺̜̾ ı̺̜̿ ı̺̺̝ ı̻̺̝ ı̼̺̝ ı̺̝̽ ı̺̝̾ ı̺̝̿ ı̺̺̞ ı̻̺̞ ı̼̺̞ ı̺̞̽ ı̺̞̾ ı̺̞̿ ı̺̺̟ ı̻̺̟ ı̼̺̟ ı̺̟̽ ı̺̟̾ ı̺̟̿ ı̺̺̠ ı̻̺̠ ı̼̺̠ ı̺̠̽ ı̺̠̾ ı̺̠̿ ı̡̺̺ ı̡̻̺ ı̡̼̺ ı̡̺̽ ı̡̺̾ ı̡̺̿ ı̢̺̺ ı̢̻̺ ı̢̼̺ ı̢̺̽ ı̢̺̾ ı̢̺̿ ı̺̺̣ ı̻̺̣ ı̼̺̣ ı̺̣̽ ı̺̣̾ ı̺̣̿ ı̺̺̤ ı̻̺̤

Ā ā Ă ă Ą ą Ć ć Ĉ ĉ Ċ ċ Č č Ď ě Đ đ Ē ē Ĕ ĕ Ė ė Ę ę Ě ě

Ğ ğ Ğ ğ Ğ ğ Ĥ ĥ Ħ ħ Ĩ ĩ Ī ī Ĭ ĭ Į į Ĳ ĳ Ĵ ĵ Ķ ķ

GREEK:  $\tau\mu$ -m BOTS

◆ B L Δ ∈ ℑ ▣ Θ ∓ κ Λ ✂ ⚡ ☾ Ⓚ Π ♣ Σ ⊥ ∧ Φ × Ψ Ω

α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω

CAUTION: Form Bets

АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ

абвгдежзийклмнопрстуфхцчшщъыьэюя



# Printable ASCII 0m3n

ASCII (00)

The quick brown fox jumps over the lazy dog  
0123456789 AbCdE {[(>)]} !@#\$\$%

ASCII (00)

The quick brown fox jumps over the lazy dog  
0123456789 AbCdE {[(>)]} !@#\$\$%

MEDIA (00)

The quick brown fox jumps over the lazy dog  
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

MEDIA ASCII (00)

The quick brown fox jumps over the lazy dog  
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

BORD (700)

The quick brown fox jumps over the lazy dog  
0123456789 AbCdE {[(>)]} !@#\$\$%

BORD ASCII (700)

The quick brown fox jumps over the lazy dog  
0123456789 AbCdE {[(>)]} !@#\$\$%

EXTRA-BORD (000)

The quick brown fox jumps over the lazy dog  
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

EXTRA-BORD ASCII (000)

The quick brown fox jumps over the lazy dog  
0123456789 AaBbCcDd {[(>)]} !@#\$\$%

SIZE COMPILED: ASCII 0m3n

The quick brown fox jumps over the lazy dog

$\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \frac{d}{dt} u dx$   
 $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \frac{d}{dt} u dx$   
 $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \frac{d}{dt} u dx$   
 $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \frac{d}{dt} u dx$

$$\| \text{la-0+ai}^{\text{ro}} \|_{W^{1,2}(\mathbb{R}^n)} \leq C \| \text{la-0+ai}^{\text{ro}} \|_{L^2(\mathbb{R}^n)} \quad \text{for } 0 \leq \gamma \leq 1$$

$$\| \text{la}^{-Q} \text{ai}^Q \|_{W^{1,p}(\mathbb{R}^n)} \leq C \| \text{la}^{-Q} \text{ai}^Q \|_{L^p(\mathbb{R}^n)} \quad \text{for } p \in [1, \infty).$$

$$\| \lambda^{-\alpha} a_i r^0 \|_{X^{s,p}} = C_m / (\lambda^{\alpha}) / \| \lambda^{-\alpha} a_i r^0 \|_{L^\infty(\mathbb{R}^n)} \quad 0 \leq \alpha < s$$

## ERG on $x$ conc

$\frac{d}{dt} \left( \int_{\Omega} u^2 dx \right) = -2 \int_{\Omega} u \Delta u dx + 2 \int_{\partial \Omega} u \frac{\partial u}{\partial n} dS$

NI              

## LIFE TYPE



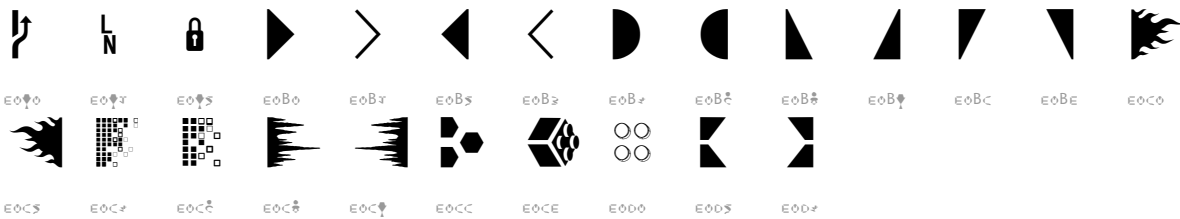
DE/EGOMEDIA



▲◆⊥⊕◆



BOERHIE


$$p_{la^{-0}ai^0} = \frac{1}{2} \left( 1 + \frac{C_{m/l}^0}{p_{la^{-0}ai^0}} \right) \cdot 0.675$$

## Provenance & Concept

## Source code

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>612 <math>\mu</math>ono</p> | <p>o i d i l o f i r . r a t i m i -<br/> l i l i l i f i c c u r i i l l a y a . p l a t a i t a t a<br/> i t l a t t ( - , a - ) , i i i t o - , a n d t a t a t a r a t i - /<br/> f o o / 6 y i l l i ( e o l y r t r m b e r s t a o r - y r t<br/> f o o i a c o a t o l i r c r t t o o i d f l y<br/> i a m i d a t i - w i t h c i r c l e ( f a l t ) a n d r o c t a d l<br/> ( o r ) v a i a t a .</p> |
| <p>3C</p>                      | <p>c r i a i m p i - , y o c o o i d o f r a r c o c -<br/> f i l i t a t a f r o t a r c t r o : f r c t a t i - , y m t l ,<br/> m i c a , a n d e n t i t a t i - .</p>                                                                                                                                                                                                                                         |
| <p>612 <math>\mu</math>ono</p> | <p>y a o x c r t y r o , i c r a c i n d f r i o c . r s , o o t<br/> o l r i c i c l i d w o r l i o , e n t w a t m ,<br/> o i c , m a t r i a l o i d , a n d m r o , a l l i n l l d v i a<br/> t a a e n t t a t a .</p>                                                                                                                                                                                      |

**ህዝብ ምክር ቤቱ**

[illegible]

## ence

plasma is a plasma and the temperature is not constant (see 1.1).

for all - we're good, miffed at it, and re-education for  
inclusion in commercial work, find that miffed at it  
not a lot of time and carry a different name.

Carry all-ways

- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$
- $\frac{1}{2} \frac{d}{dt} \int_{\mathbb{R}^n} |u|^2 dx = \int_{\mathbb{R}^n} u \Delta u dx = - \int_{\mathbb{R}^n} |\nabla u|^2 dx \leq 0$